

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Task 1: Project Completion and Final Implementation

Requirements refinement (not scope change)

- NFR-09's cold-start threshold was changed from a placeholder of <2000ms to <500ms once real data (~75-100ms typical cold-start) showed the original threshold was too lenient.

```
//NFR-09: cold-start (model load + first inference) latency, tracked separately
if (coldStartLatencyMs.HasValue)
{
    report.NfrResults.Add(new NfrCheckResult
    {
        RequirementId = "NFR-09",
        Description = "Cold-start latency (model load + first inference)",
        Passed = coldStartLatencyMs.Value < 500.0, //not placeholder anymore, data has been analysed
        ActualValue = $"{coldStartLatencyMs.Value:F2}ms",
        Threshold = "<500ms"
    });
}
```

- NFR-02's threshold (≥ 15 FPS for 95% of frames) was implemented and tested against real data, revealing the system always achieves ~10-12 FPS under baseline CPU-only inference which is an NFR failure, not an implementation gap/ This brings the question of whether 15 FPS is a realistic target for CPU-bound embedded-simulated inference, which we will revisit during quality metrics and release evaluation (Task 8) rather than treat it as a defect to silently fix by lowering the threshold.

The screenshot shows the Test Explorer window with a list of tests on the left and a detailed view of a failed test on the right. The test 'Generate_WithLargerFrameBatch_EvaluatesNfr02FrameRateCompliance' has failed. The standard output shows frame rates for 20 frames, all of which are below the 15 FPS threshold.

Frame	Time (ms)	FPS
1	89.25ms	11.2
2	89.92ms	11.1
3	85.48ms	11.7
4	87.58ms	11.4
5	88.81ms	11.4
6	91.21ms	11.0
7	85.96ms	11.6
8	89.23ms	11.2
9	84.96ms	11.8
10	81.86ms	12.3
11	98.89ms	11.1
12	85.76ms	11.7
13	96.37ms	10.4
14	83.21ms	12.0
15	98.71ms	11.0
16	81.18ms	12.3
17	93.27ms	10.7
18	88.86ms	11.4
19	98.53ms	11.0
20	98.22ms	10.2

NFR-02: Failed - At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames ≥ 15 FPS, threshold: $\geq 95\%$ of frames ≥ 15 FPS)

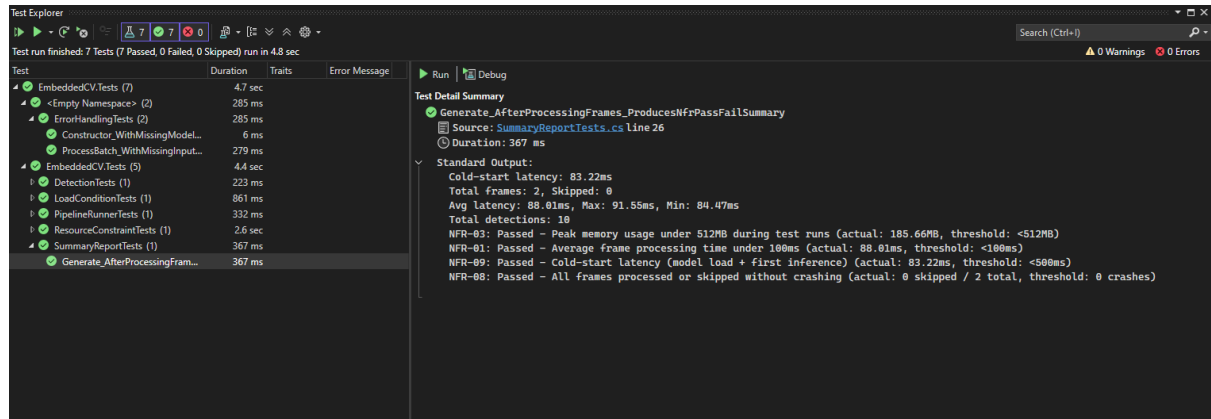
Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

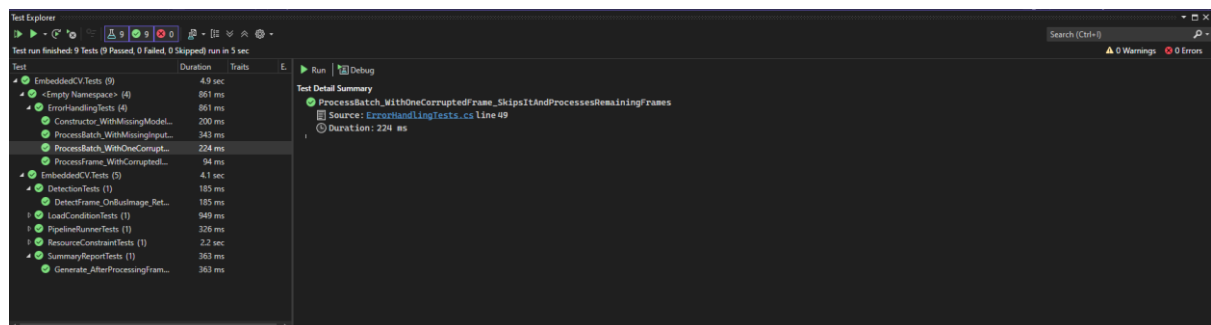
Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Implementation additions (filling gaps seen in Assessment 1 feedback)

- NFR-03 (memory usage) previously had a defined threshold with no implemented measurement; this has been implemented using process working set memory sampling and is now actively tested



- NFR-08 (error handling) previously only verified that no unhandled exception occurred during a run; this has been changed with four dedicated test cases validating each specific error condition individually (missing model, corrupted frame, invalid input path), addressing the “false pass” risk seen in Assessment 1 feedback. Under ErrorHandlingTests are the four tests, covering Missing model (FR-09), Missing/invalid input path (FR-11), Single corrupted frame isolated (FR-10), and corrupted frame within a batch, confirming continuation (FR-10).



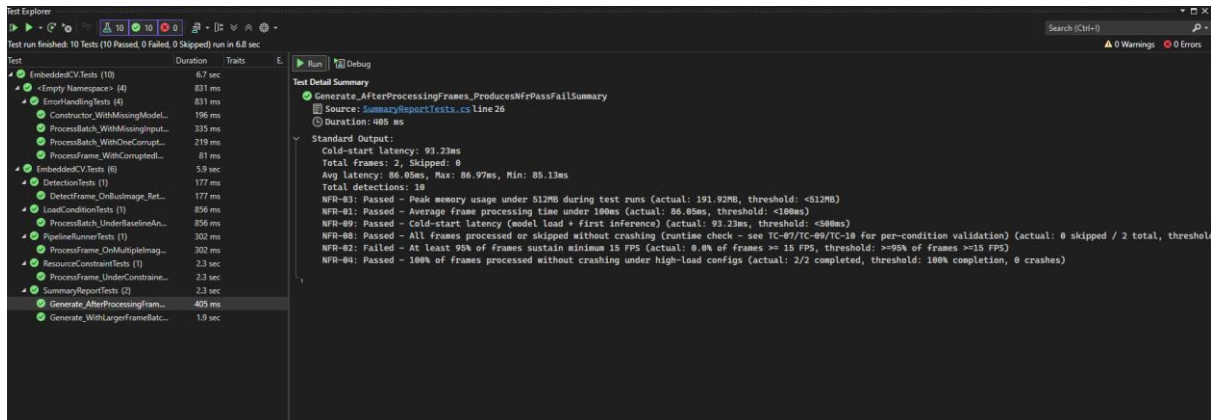
Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

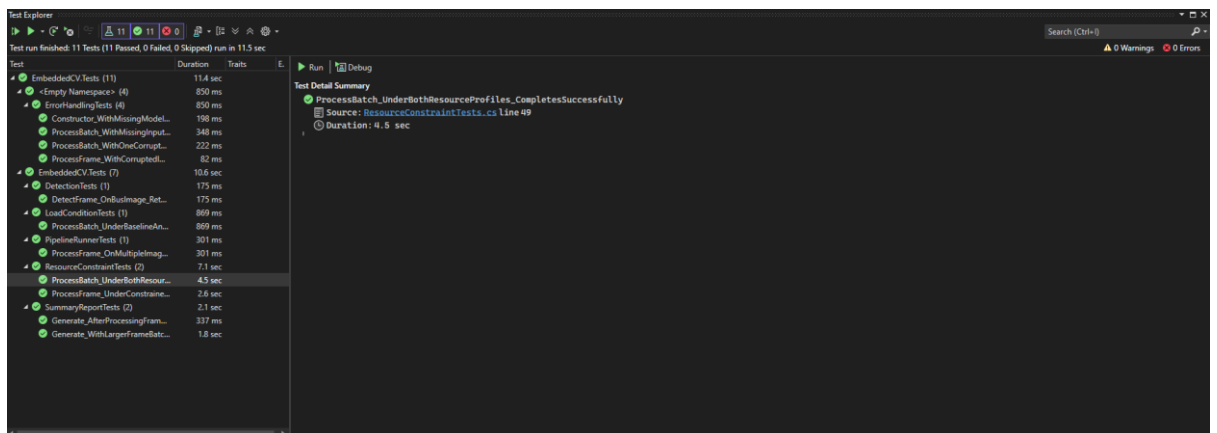
Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

- NFR-04, NFR-06, and NFR-07 previously untested, now have dedicated automated test coverage confirming reliability under load, consistency across repeated runs, and portability across hardware profiles.

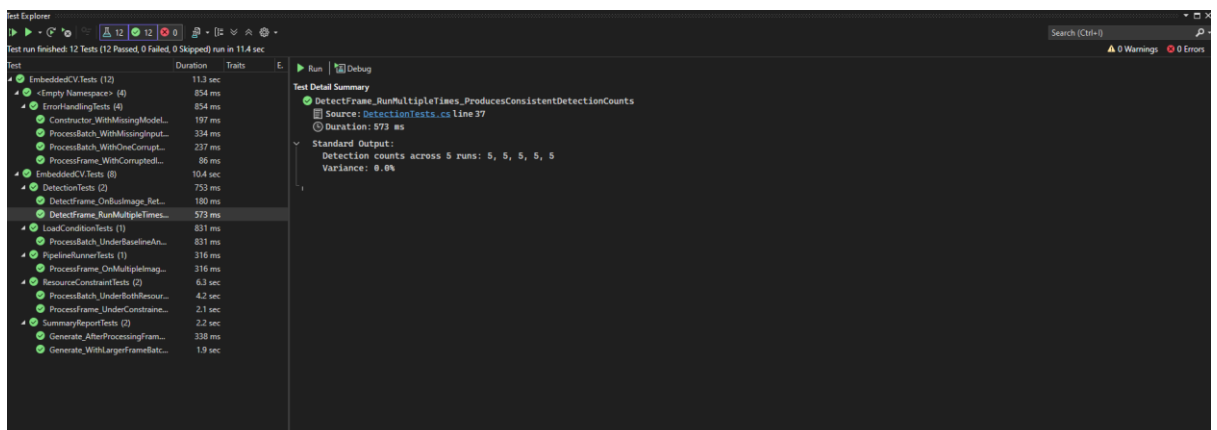
NFR-04



NFR-06



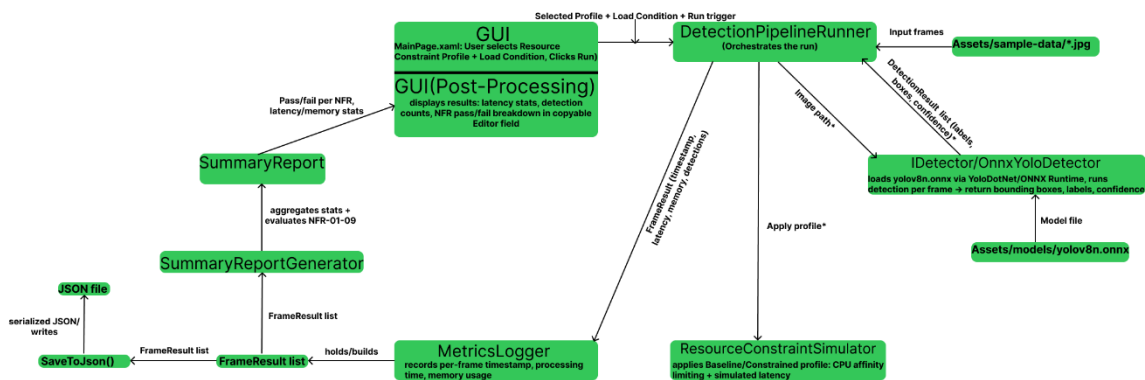
NFR-07



Architectural or technological changes

The core architecture (GUI → DetectionPipelineRunner → Detector/ResourceConstraintSimulator/MetricsLogging → SummaryReportGenerator) and the stack (C#/.NET 9, MAUI Windows, YoloDotNet/ONNX Runtime, MSTest) are unchanged from Assessment 1. Development since then is focused on finishing test coverage and measurement gaps seen in feedback, not restructuring the system.

Architecture/Data Flow diagram



*: For the setup of those marked, Image path and DetectionResult list are per-frame, repeated calls
But Apply profile is a one-time setup call (before the per-frame loop begins)

GUI(Profile: Baseline, Load condition: Baseline)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Baseline

Load Condition

Select load condition

Baseline

Run Test

Run complete.

Results

Profile: Baseline | Load: Baseline

Frames: 2 (Skipped: 0)
Avg latency: 87.84ms
Max latency: 88.16ms
Min latency: 87.52ms
Total detections: 10

NFR Results:

NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 300.27MB, threshold: <512MB)
NFR-01: PASS — Average frame processing time under 100ms (actual: 87.84ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

GUI(Profile: Baseline, Load condition: High load)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Baseline

Load Condition

Select load condition

High Load

Run Test

Run complete.

Results

Profile: Baseline | Load: HighLoad

Frames: 2 (Skipped: 0)
Avg latency: 136.57ms
Max latency: 164.49ms
Min latency: 108.65ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 312.88MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 136.57ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

GUI(Profile: Constrained, Load condition: Baseline)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Constrained (Embedded-like)

Load Condition

Select load condition

Baseline

Run Test

Run complete.

Results

Profile: Constrained (Embedded-like) | Load: Baseline

Frames: 2 (Skipped: 0)
Avg latency: 2673.44ms
Max latency: 2980.70ms
Min latency: 2366.18ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 318.76MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 2673.44ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

GUI(Profile: Constrained, Load condition: High load)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Constrained (Embedded-like)

Load Condition

Select load condition

High Load

Run Test

Run complete.

Results

Profile: Constrained (Embedded-like) | Load: HighLoad

Frames: 2 (Skipped: 0)
Avg latency: 3917.87ms
Max latency: 4402.50ms
Min latency: 3433.25ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 321.32MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 3917.87ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Deferred, not implemented

A feature, considered previously, which would allow users to upload their own dataset for testing (rather than the fixed sample set) was evaluated and deliberated on by Me (Yaacoub) and Matthew, we concluded that it doesn't map to any required Assessment 2 task and was judged not a good use of our time. This is documented as a future recommendation (Task 10) rather than abandoned.

NFR-05 (configurability) hasn't been automated, since it's fundamentally a GUI usability requirement rather than backend logic suited to unit testing. It'll be validated as a manual usability test case under Task 7 (Quality Testing), supported by GUI screenshots.

Task 2: Requirements Verification and Traceability

- Review and finalise the functional and non-functional requirements from Assessment 1.
- Update the Requirements Traceability Matrix.
- Map requirements to implemented features, test cases, test results, and defects where applicable.
- Identify any requirements that are partially implemented, unverified, or out of scope, and justify them.

Task 3: Test Execution and Final Test Results

- Execute the planned tests against the completed system.
- Include relevant functional and non-functional tests, such as unit, integration, system, acceptance, regression, usability, security, performance, or accessibility testing.
- Include positive, negative, boundary, invalid-input, and error-handling scenarios where applicable.
- Provide at least *fifteen (15)* final test cases with execution results.
- Summarise the number of planned, executed, passed, failed, blocked, and not-run tests.
- Explain failed tests, corrective actions, regression testing, and remaining quality risks.

Test Case Table

ID	Test Case	Preconditions	Steps	Input	Expected Result	Requirements	Type	Priority	Status
TC-01	Detector returns valid detections on known sample image	Model + sample image available	1. Load detector 2. Call DetectFrame(bus.jpg)	bus.jpg	Returns list of detections with labels, confidence, bounding box	FR-01-04	Unit/Integration	High	Pass
TC-02	Detection results consistent across repeated runs	Model loaded, warmed up	1. Run DetectFrame 5x on same image	bus.jpg x5	Detection count variance $\leq 2\%$	NFR-06	Non-functional	Medium	Pass (0% variance)
TC-03	Detector returns empty list on blank image	Model loaded	1. Generate blank image 2. Run DetectFrame	Blank white image	Returns empty detection list, no error	FR-02, FR-03	Boundary	Medium	Pass
TC-04	Full pipeline logs results and timing for multiple frames	Model + 2 sample images available	1. Process bus.jpg, frame 1 2. Process zidane.jpg, frame 2	bus.jpg, zidane.jpg	2 FrameResults logged with valid detections, timestamps, latency	FR-04, FR-05	Integration	High	Pass
TC-05	Logged results export to valid, parsable JSON	Frames processed and logged	1. Call SaveToJson() 2. Deserialize output	Logged FrameResult list	File exists, deserializes without error, matches frame count	FR-05	Integration	Medium	Pass
TC-06	Constrained resource profile produces higher latency than baseline	Model loaded	1. Run under Baseline profile 2. Run under Constrained profile	bus.jpg, both profiles	Constrained latency measurably higher	FR-06, NFR-01	Non-functional	High	Pass
TC-07	System completes a full run under both resource profiles	Model + sample images available	1. Run batch under Baseline 2. Run batch under Constrained	bus.jpg, zidane.jpg	Both runs complete, 0 frames skipped	FR-07, NFR-07	Non-functional	High	Pass
TC-08	Summary report evaluates latency (NFR-01) pass/fail against threshold	Frames processed	1. Generate summary report	Logged FrameResults	NFR-01 result reflects accurate pass/fail vs 100ms threshold	FR-07, NFR-01	Non-functional	High	Pass
TC-09	Frame rate compliance (NFR-02) evaluated across larger batch	Model warmed up	1. Process 20-frame batch 2. Generate report	20 images (repeated samples)	NFR-02 result reflects real % of frames ≥ 15 FPS	FR-07, NFR-02	Non-functional	Medium	Fail (0% compliance, system achieves ~11 FPS avg)
TC-10	Cold-start latency measured separately from steady-state	Fresh detector instance	1. Measure first DetectFrame call 2. Process further frames	bus.jpg	Cold-start and steady-state latency reported separately	FR-07, NFR-09	Functional	Medium	Pass
TC-11	Batch processing completes under Baseline and HighLoad conditions	Model + sample images available	1. Run batch, Baseline load 2. Run batch, HighLoad	4 images	Both complete all frames, 0 skipped	FR-08	Negative	High	Pass
TC-12	Detector constructor throws clear error on missing model	No model file at given path	1. Construct OnnxYoloDetector with invalid path	Invalid model path	FileNotFoundException thrown with clear message	FR-09	Negative	High	Pass
TC-13	Batch run rejected before execution on missing input file	Model available	1. Call ProcessBatch with non-existent image path	Invalid image path	FileNotFoundException thrown, no frames processed	FR-11	Negative	High	Pass
TC-14	Corrupted frame is skipped and logged, run continues	Model available, 1 corrupted + 1 valid file	1. Process corrupted file alone 2. Process batch with corrupted + valid file	Corrupted file, bus.jpg	Corrupted frame marked skipped with reason; valid frame still processed	FR-10	Negative	High	Pass
TC-15	Empty batch request is rejected before execution	Model available	1. Call ProcessBatch with empty list	Empty list	ArgumentException thrown	FR-11	Boundary/Negative	Medium	Pass

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Execution Summary

Metric	Count
Planned	15
Executed	15
Passed	14
Failed	1 (TC-09)
Blocked	0
Not Run	0

Explanation of Failed Test and Corrective Action

TC-09 (NFR-02, frame rate compliance) is the only failed test case, and it fails by design rather than a defect. The test method itself runs properly and accurately measures real system behaviour. The system under test doesn't meet the 15 FPS threshold, consistently getting ~10-12 FPS on CPU-only YOLOv8n inference across a 20-frame sample. This is being treated as an actual, informative NFR failure rather than a bug to patch, since it demonstrates the real-world software quality problem this project sought out to investigate (Functional correctness does not guarantee real-time performance compliance). The corrective action under consideration is not a code fix, but a requirement-level discussion whether the 15 FPS threshold is realistic for CPU-only inference in an embedded-simulated context. It's to be resolved during quality metrics and release evaluation (Task 8). No regression risk exists from this result, since the failure shows measured hardware/software performance, not bad logic.

Regression Testing

The full MTest suite is re-run after every code change via Visual Studio's Test Explorer, following the regression practice after the Assessment 1 concurrency bug (where parallel test execution caused leakage between tests). [assembly: DoNotParallelize] stays to guarantee consistent, repeatable results across runs, and all 15 test cases are re-verified as part of this regression process before each commit.

Remaining Quality Risks

- TC-09's result raises a question about NFR-02's threshold, to be resolved in Task 8 rather than unresolved
- NFR-05 (configurability) is still validated manually rather than through automated testing (See Task 7), which holds a small risk of being under-documented compared to the automated suite.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

- Test coverage relies on a small, fixed sample image set (2 base images, reused/repeated for volume), which lowers confidence in detection-accuracy claims across real-world conditions.

Task 4: Test Automation and Quality Engineering

- Identify suitable test cases for automation and justify the selection.
- Implement automated tests using an appropriate tool or framework where suitable.
- Report automated test execution results using logs, screenshots, reports, or repository evidence.
- Discuss the benefits, limitations, and maintainability of the automated test suite.
- Where automation is not appropriate, justify the use of manual or exploratory testing.

Task 5: Continuous Integration and DevOps QA

- Describe the development and integration workflow.
- Implement a simple CI workflow or equivalent automated quality check where appropriate.
- Define quality gates such as successful build, passing tests, linting, code review, or coverage threshold.
- Demonstrate smoke or sanity testing where relevant.
- If CI/CD is not technically appropriate, justify an alternative quality workflow.

Task 6: Defect Management and Root Cause Analysis

- Maintain a final defect register covering defects found during development and testing.
- Define severity, priority, and defect status categories.
- Describe the defect lifecycle used in the project.
- Analyse at *least three (3)* significant defects using root cause analysis.
- Propose corrective actions and defect-prevention measures.
- Explain how defect analysis improved product or process quality.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Task 7: Quality Testing

- Conduct quality testing relevant to the project, such as security, performance, usability, accessibility, compatibility, reliability, or maintainability testing.
- Address at *least two (2)* quality areas where applicable.
- Use risk-based reasoning to justify the selected testing activities.
- Document testing evidence, results, limitations, and recommended improvements.

Task 8: Quality Metrics and Final Release Decision

- Present relevant quality metrics, such as test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, or commit activity.
- Evaluate the completed system against requirements, acceptance criteria, and quality attributes.
- Summarise testing results, defect status, metrics, limitations, and residual risks.
- State whether the system is ready for release, conditionally ready, or not ready for release.
- Justify the release decision using evidence.

Task 9: AI-Assisted Development and Critical Reflection

- Explain how GitHub Copilot was used for implementation, testing, debugging, refactoring, or QA activities.
- Provide representative evidence of AI-assisted development.
- Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected.
- Identify any quality risks found in AI-assisted outputs, such as incorrect logic, insecure code, poor maintainability, or unsuitable assumptions.
- Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice.

Task 10: Quality Improvement and Future Recommendations

- Identify the main quality lessons learned from the project.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

- Recommend improvements to the development and QA process.
- Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability.
- Explain how the project could evolve toward stronger continuous quality engineering practice.

Task 11: Team Contribution and Professional Reflection

- Provide a contribution statement for each group member.
- Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks.
- Include a brief individual reflection from each group member.

Task 12: Project Presentation and Demonstration

Each group must prepare a 15-slide presentation and demonstrate the completed project. The presentation should clearly summarise the problem, final solution, quality assurance activities, testing evidence, and final quality evaluation. The presentation should cover:

- Project title, group members, problem, stakeholders, and scope
- Key functional and non-functional requirements
- Final prototype, implemented features, and changes after Assessment 1
- Testing approach, coverage, execution results, and key findings
- Automation, CI/CD, or equivalent quality workflow
- Defects, root cause analysis, and corrective actions
- Quality metrics, advanced testing, and release decision
- GitHub Copilot use, limitations, risks, and future improvements
- Individual contributions and lessons learned.

All students must participate in the demonstration and show the working prototype or quality improvement solution. The demo should include key features, important test cases, automated tests or CI results where applicable, defect fixes, and evidence that major requirements have been satisfied. Students must be prepared to answer questions about the project, code, testing, quality artefacts, AI-assisted development, and individual contributions.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Deliverables

- Final Project and Software Quality Assurance Report: *6000--7000 words*
- Source code and GitHub repository link; provide evidence from GitHub commits.
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report